

Advanced Programming

Project assignment – 3rd stage

JavaFX

MVC@PA

Stage 3 – *JavaFX*

- All students must *update project* and *checkout* the `main` *branch*
- The student responsible for this task should start by creating a new *branch* from the `main` *branch*
 - *Branch* name: **stage3**
- The remaining students should update the project and then *checkout* the *branch*

Stage 3

- Deadline to perform the *Pull Request* and the *Squash and Merge* (applies to all groups, including working students):

***** 2025.04.22 23:59 *****

- The *Squash and Merge* into the *main* branch must be performed on the *GitHub* website
- The branch must not be deleted after the *Merge* is completed
- The next stage will be presented in a Zoom session to be held on: **2025.04.23 16:30**
 - Link: <https://videoconf-colibri.zoom.us/j/93423788633>
 - The session will be recorded

Stage 3

- Integrate *JavaFX* into the project
- Create and configure a base window to support the chess game's UI, following the MVC@PA organization taught in the theoretical classes
- At this stage, the drawing of the chessboard and its pieces should not yet be included

Stage 3

- Provide an application menu that allows the following options:
 - Start a new game, load a saved game, save the current game, import and export partial games
 - Learning mode options, such as: undo and redo (for future implementation)
- When a new game is created, player names must be requested and stored in the data model – `ChessGame` class

JavaFX installation

- The most up-to-date documentation can be obtained from the *OpenJFX* website
 - <https://openjfx.io/>
- Integrating *JavaFX* into a *Java* project can be done by downloading the library and associating it with the project
 - The library consists of several *jar* files

JavaFX installation

- Download from the *website* : <http://openjfx.io>
 - *Direct link* : <https://gluonhq.com/products/javafx/>
- For new projects, *download* an *LTS version* (21.0.6) or the latest version (24)
 - Download...
 - The *SDK* for the desired operating system
 - The documentation (*JavaDoc*)
- Unzip the files to a directory, preferably near the *JDK* in use (the documentation can be unzipped into a `doc` directory within the created base directory)
 - *Windows* : `C:\Program Files\Java`
 - *macOS* : `/Library/Java/JavaVirtualMachines`

JavaFX IntelliJ configuration

- Go to *File* → *Project Structure*
 - Select *Global Libraries*
 - Click "+" → *New Java Library ...*
 - Enter the full path to the *JavaFX SDK*
- **Set the library name to: PAJavaFX**
- Add the documentation *URLs* to the library
 - *URL* examples for local files:
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc`
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc\javafx.base`
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc\javafx.controls`
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc\javafx.fxml`
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc\javafx.graphics`
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc\javafx.media`
- These steps need to be performed only once

JavaFX configuration

- In the *Java* project where *JavaFX* development is intended to start:
 - Go to *File* → *Project Structure*
 - Select *Global Libraries*
 - Right-click on the library "**PAJavaFX**" and "*Add to modules...*", then add it to the active project/module

JavaFX project configuration

- In the run configurations
 - Add the following settings to the *VM Options*
 - `--module-path "/path_to_javafx_sdk/lib"`
 - `--add-modules javafx.controls`
 - if *xml* is used for the interface definition:
 - `--add-modules javafx.controls,javafx.fxml`
 - to include all the *JavaFX* modules
 - `--add-modules ALL-MODULE-PATH`

JavaFX with Maven

- Alternatively, *JavaFX* can be included in the project using *Maven*
 - Note that these instructions do not aim to alter the entire structure of the project to follow the usual *Maven* project model
- Include a file named `pom.xml` at the root of the project
 - When creating this file, *IntelliJ* will usually detect the presence of the *Maven* framework and automatically offer to load the configuration – confirm this request (bottom right corner)
 - Include the content from the following slide

pom.xml (1/2)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>pt.isec.pa</groupId>
  <artifactId>chess</artifactId>
  <version>stage3</version>

  <properties>
    <maven.compiler.source>23</maven.compiler.source>
    <maven.compiler.target>23</maven.compiler.target>
    <maven.compiler.encoding>UTF-8</maven.compiler.encoding>
    <javafx.version>23.0.1</javafx.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.openjfx</groupId>
      <artifactId>javafx-controls</artifactId>
      <version>${javafx.version}</version>
    </dependency>
    <dependency>
      <groupId>org.openjfx</groupId>
      <artifactId>javafx-graphics</artifactId>
      <version>${javafx.version}</version>
    </dependency>
  </dependencies>
```

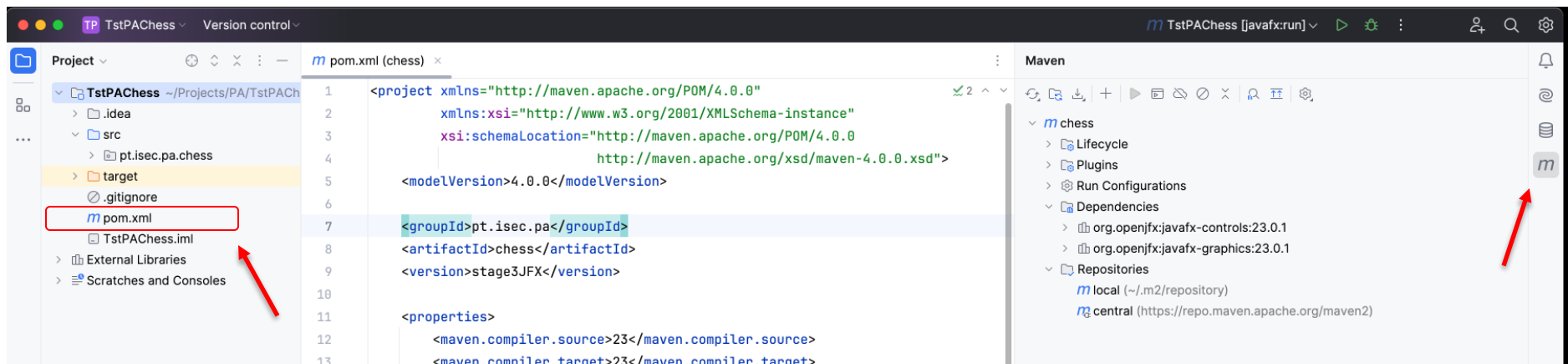
...

pom.xml (2/2)

```
...
    <build>
      <sourceDirectory>src</sourceDirectory>
      <resources>
        <resource>
          <directory>src</directory>
          <includes>
            <include>pt/isec/pa/chess/ui/res/**</include>
          </includes>
        </resource>
      </resources>
      <plugins>
        <plugin>
          <groupId>org.openjfx</groupId>
          <artifactId>javafx-maven-plugin</artifactId>
          <version>0.0.8</version>
          <configuration>
            <mainClass>pt.isec.pa.chess.ChessApp</mainClass>
          </configuration>
        </plugin>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-resources-plugin</artifactId>
          <version>3.3.1</version>
          <configuration>
            <encoding>${maven.compiler.encoding}</encoding>
          </configuration>
        </plugin>
      </plugins>
    </build>
  </project>
```

JavaFX with Maven

- The configurations (with the necessary adaptations for the project and versions used) allow the system to fetch the required libraries from *Maven* repositories and will also enable compiling and running the *JavaFX* program
 - Note that on the right side of *IntelliJ*, a *Maven* menu will appear, from which you can trigger actions (compiling and running), and it also allows you to download the entire *JavaFX* documentation easily



- Consult the slides and examples from the theoretical classes to organize the *JavaFX* graphical interface classes according to the architectural model presented: *MVC@PA*

Project example

```
public class Main {  
    public static void main(String[] args) {  
        Application.launch(MainJFX.class, args);  
    }  
}
```

```
public class MainJFX extends Application {  
    ModelData data;  
  
    public MainJFX() { data = new ModelData(); } // It can also be created in 'init'  
  
    @Override  
    public void start(Stage stage) throws Exception {  
        RootPane root = new RootPane(data);  
        Scene scene = new Scene(root, 600, 400);  
        stage.setScene(scene);  
        stage.setTitle("TeoIntroJFX");  
        stage.show();  
    }  
}
```


Project example

```
public class RootPane extends VBox { //View-Controller
    ModelData data;
    // variables, including reference to views

    public RootPane(ModelData data) {
        this.data = data;

        createViews();
        registerHandlers();
        update();
    }

    private void createViews() { /* create and configure views */ }

    private void registerHandlers() { /* handlers/listeners */ }

    private void update() { /* update views */ }
}
```

Chess game menu

- Include a `MenuBar` in the *JavaFX* application to present the following options:
 - Menu "Game":
 - `New` – Create a new chess game
 - Users should be prompted to enter their names
 - `Open` – Restore a previously saved chess game
 - `Save` – Save the current game using serialization
 - `Import` – Import a txt/csv partial game
 - `Export` – Export a txt/csv partial game (as defined in the previous stage)
 - `Quit` – Exit the game and close the application
 - Menu "Mode"
 - `Normal` – Standard playing mode
 - `Learning` – Provides mechanisms to assist learning players, including:
 - `Show possible moves` – Check to display the possible moves for a selected piece
 - `Undo` – Undo the last move
 - `Redo` – Redo a move that was undone

Chess game menu

- The content on creating menus will be taught in the theoretical class on 2025.04.11
- The *undo* and *redo* options should be inactive at this stage
 - It will be defined how these options will be implemented in a later stage

ChessGameManager

- To control access and protect the ChessGame class, a new *Facade* should be created
 - Suggested name: ChessGameManager
 - Package: `pt.isec.pa.chess.model`
- From the *UI* perspective, this will be treated as the actual data model
- This class will maintain a reference to the active ChessGame instance, which should be created in its constructor

ChessGameManager

- The class should provide methods to grant access to all public methods of `ChessGame`
- It must not return references to the internal data model classes/instances to the *UI*
- This manager will handle all file-related operations (*Open*, *Save*, *Import*, and *Export*).
 - It should use the `ChessGameSerialization` class to handle the *Open* and *Save* operations, and
 - It should use the `ChessGame` `import` and `export` methods to manage the *Import* and *Export* of partial games
- This class will later be used to provide additional functionalities

Other developments

- Students can make use of the extra time during Easter to implement the special chess moves: *Castling*, *En Passant*, and *Pawn Promotion*
 - This is optional for this stage but should be implemented by June 2nd